

```
"""
```

```
QuadMind Scoring Engine v3.3
```

```
© BRAIN JUICE 2026
```

```
v3.2 からの修正:
```

1. strict=False の partial mode を削除。全問回答必須 (strict=True) に固定
2. G5フラグを2段階 (watch: 15点以上 / high: 20点以上) に変更
3. risk_flags の文言をビジネス診断向けに変更 (医療判断的表現を除去)

```
"""
```

```
import json
```

```
from collections import defaultdict
```

```
from typing import Any
```

```
CROSS_WEIGHT = 0.7 # クロス加点係数
```

```
RESP_THRESHOLD = 3.0 # 責任感複合型の差分閾値
```

```
G5_WATCH_THRESHOLD = 0.6 # G5 watch フラグ閾値 (15点以上)
```

```
G5_HIGH_THRESHOLD = 0.8 # G5 high フラグ閾値 (20点以上)
```

```
def load_db(path: str) -> list[dict]:
```

```
    with open(path, encoding="utf-8") as f:
```

```
        return json.load(f)
```

```
def _question_ids(db: list[dict]) -> set[str]:
```

```
    """DB内の全 question_id を返す"""
```

```
    return {row["question_id"] for row in db}
```

```
def validate_responses(db: list[dict], responses: dict[str, str]) -> list[str]:
```

```
    """
```

```
    全問回答チェック (方式A)。
```

```
    未回答の question_id のリストを返す。空なら OK。
```

```
    """
```

```
    required = _question_ids(db)
```

```
    return sorted(required - set(responses.keys()))
```

```
def score(
```

```
    db: list[dict],
```

```
    responses: dict[str, str],
```

```
) -> dict[str, Any]:
```

```
    """
```

Parameters

db : load_db() で読み込んだ採点DB

responses : { question_id: option }

全問回答必須。未回答があれば ValueError を送出する。

部分回答を処理したい場合は validate_responses() で事前確認し、
回答済み問のみのサブセットDBを渡すこと。

"""

— 1. バリデーション（全問必須） —————

missing = validate_responses(db, responses)

if missing:

raise ValueError(

f"未回答の質問があります ({len(missing)}問) : {missing}\n"

"全問回答してから再実行してください。"

)

— 2. 累積変数の初期化 —————

raw_target = defaultdict(float)

raw_pref = defaultdict(float)

raw_low = defaultdict(float)

raw_obs = defaultdict(float)

neutral_count = 0

diagnostic_null_count = 0

neutral_by_axis = defaultdict(int)

— 3. pass1: 正規化分母（質問単位・最大値方式） —————

_tc_max = defaultdict(lambda: defaultdict(float))

_le_max = defaultdict(lambda: defaultdict(float))

_ob_max = defaultdict(lambda: defaultdict(float))

_pref_max = defaultdict(lambda: defaultdict(float)) # Preference正規化用

for row in db:

target_axis = row["target_axis"]

question_id = row["question_id"]

primary_axis = row["primary_axis"]

weight = row["weight"]

target_credit = row["target_credit"]

low_evidence = row["low_evidence"]

is_neutral = row["is_neutral"]

is_diagnostic_null = row["is_diagnostic_null"]

is_observer = row["is_observer"]

if is_neutral:

continue

if is_observer:

```

        v = weight * target_credit
        if v > _ob_max[target_axis][question_id]:
            _ob_max[target_axis][question_id] = v
    else:
        vt = weight * target_credit
        if vt > _tc_max[target_axis][question_id]:
            _tc_max[target_axis][question_id] = vt
        vl = weight * low_evidence
        if vl > _le_max[target_axis][question_id]:
            _le_max[target_axis][question_id] = vl
        # Preference Score の分母: primary_axis ごとに質問単位最大値
        if primary_axis is not None:
            pv = weight * CROSS_WEIGHT
            if pv > _pref_max[primary_axis][question_id]:
                _pref_max[primary_axis][question_id] = pv

raw_target_max = {ta: sum(qm.values()) for ta, qm in _tc_max.items()}
raw_low_max    = {ta: sum(qm.values()) for ta, qm in _le_max.items()}
raw_obs_max    = {ta: sum(qm.values()) for ta, qm in _ob_max.items()}
raw_pref_max   = {pa: sum(qm.values()) for pa, qm in _pref_max.items()}

# — 4. pass2: 回答データを採点 —————
for row in db:
    question_id      = row["question_id"]
    option           = row["option"]

    if question_id not in responses:
        continue
    if responses[question_id] != option:
        continue

    target_axis      = row["target_axis"]
    primary_axis     = row["primary_axis"]
    target_credit    = row["target_credit"]
    low_evidence     = row["low_evidence"]
    weight           = row["weight"]
    is_reverse       = row["is_reverse"]
    is_observer      = row["is_observer"]
    is_neutral       = row["is_neutral"]
    is_diagnostic_null = row["is_diagnostic_null"]

    if is_diagnostic_null:
        diagnostic_null_count += 1
        raw_target[target_axis] += weight * target_credit
        raw_low[target_axis]    += weight * low_evidence
        # primary_axis には加点しない

```

```

elif is_neutral:
    neutral_count += 1
    neutral_by_axis[target_axis] += 1

elif is_observer:
    raw_obs[target_axis] += weight * target_credit

else:
    raw_target[target_axis] += weight * target_credit
    if low_evidence > 0:
        raw_low[target_axis] += weight * low_evidence
    if primary_axis is not None:
        raw_pref[primary_axis] += weight * CROSS_WEIGHT

# — 5. スコア正規化 —————
MAIN_AXES = ["A", "B", "C", "D"]

def norm25(raw: float, denom: float) -> float:
    return round(25.0 * raw / denom, 2) if denom > 0 else 0.0

axis_score = {ax: norm25(raw_target[ax], raw_target_max.get(ax, 1))
               for ax in MAIN_AXES}

# Preference Score: 25点満点に正規化
preference_score = {
    k: norm25(v, raw_pref_max.get(k, 1))
    for k, v in raw_pref.items()
}

low_evidence_index = {ax: norm25(raw_low[ax], raw_low_max.get(ax, 1))
                      for ax in MAIN_AXES}

ob_denom      = raw_obs_max.get("OB", 1)
sw_denom      = raw_obs_max.get("SW", 1)
observer_score = norm25(raw_obs.get("OB", 0), ob_denom)
switching_score = norm25(raw_obs.get("SW", 0), sw_denom)
integration_index = round((observer_score + switching_score) / 2, 2)

ia_denom = raw_target_max.get("iA", 1)
ea_denom = raw_target_max.get("eA", 1)
internal_a = norm25(raw_target.get("iA", 0), ia_denom)
expressive_a = norm25(raw_target.get("eA", 0), ea_denom)

# — 6. 責任感タイプ判定 —————
resp_axes = ["DR", "BR", "AR"]
resp_label = {"DR": "D型（規範準拠）", "BR": "B型（期待応答）", "AR": "A型（結果自律）"}
resp_scores = {}

```

```

for ra in resp_axes:
    d = raw_target_max.get(ra, 1)
    resp_scores[ra] = norm25(raw_target.get(ra, 0), d)

sorted_resp = sorted(resp_scores.items(), key=lambda x: -x[1])
primary_resp = sorted_resp[0][0]
secondary_resp = sorted_resp[1][0]
resp_gap = sorted_resp[0][1] - sorted_resp[1][1]
if resp_gap < RESP_THRESHOLD:
    resp_type = f"複合型 ({resp_label[primary_resp]} × {resp_label[secondary_resp]})"
else:
    resp_type = resp_label[primary_resp]

responsibility = {
    "scores":      {ra: resp_scores[ra] for ra in resp_axes},
    "primary":     primary_resp,
    "type_label":  resp_type,
    "is_composite": resp_gap < RESP_THRESHOLD,
}

```

— 7. G5 内部フラグ判定 —————

```

g5_axes = ["AG", "RV", "IM"]
g5_label = {
    "AG": "承認略奪型傾向",
    "RV": "ルール暴力型傾向",
    "IM": "衝動暴走型傾向",
}

g5_scores = {}
for ga in g5_axes:
    d = raw_target_max.get(ga, 1)
    g5_scores[ga] = norm25(raw_target.get(ga, 0), d)

g5_watch_pts = 25.0 * G5_WATCH_THRESHOLD # 15.0
g5_high_pts = 25.0 * G5_HIGH_THRESHOLD # 20.0
g5_flags = {}
for ga in g5_axes:
    sc = g5_scores[ga]
    if sc >= g5_high_pts:
        level = "high"
    elif sc >= g5_watch_pts:
        level = "watch"
    else:
        level = "none"
    g5_flags[ga] = {
        "score": sc,
        "level": level, # "none" / "watch" / "high"
        "flagged": level != "none",
    }

```

```

        "label": g5_label[ga],
    }
active_g5 = [ga for ga in g5_axes if g5_flags[ga]["flagged"]]

# — 8. type_flags (構造化タイプ判定) —————
lei = low_evidence_index
ax = axis_score

# A状態の判定
fz_score = (
    norm25(raw_target.get("FZ", 0), raw_target_max.get("FZ", 1))
    if raw_target_max.get("FZ", 0) > 0 else 0.0
)
if internal_a >= 13 and expressive_a < 13 and fz_score >= 13:
    a_state = "A凍結型"
elif internal_a >= 13 and expressive_a < 13 and ax["B"] >= 13:
    a_state = "A抑圧型"
elif internal_a < 10 and expressive_a < 10:
    a_state = "真性A低"
elif internal_a >= 13 and expressive_a >= 13:
    a_state = "A管理型"
else:
    a_state = "標準"

# 統合状態の判定
if integration_index >= 20 and all(ax[x] >= 13 for x in MAIN_AXES):
    integration_state = "統合型 (Lv.3) "
elif integration_index >= 20:
    integration_state = "部分統合"
elif integration_index < 15 and all(14 <= ax[x] <= 18 for x in MAIN_AXES):
    integration_state = "偽の中庸"
elif integration_index < 15:
    integration_state = "単独運転"
else:
    integration_state = "標準"

# 置換パターンの検出
replacement_flags = []
if lei["C"] >= 15:
    replacement_flags.append("C→A置換 (C判断がA感情に置換) ")
if lei["D"] >= 15:
    replacement_flags.append("D→A置換 (D計画性がA衝動に置換) ")
if lei["A"] >= 15:
    replacement_flags.append("A抑制 (A表出がB/Dに置換) ")
if lei["B"] >= 15:
    replacement_flags.append("B→D置換 (B感情がD論理化に置換) ")

```

```

# リスクフラグ
risk_flags = []
if a_state == "A凍結型":
    risk_flags.append("高ストレス反応の可能性。追加面談を推奨")
for ga in active_g5:
    lvl = g5_flags[ga]["level"]
    lbl = g5_flags[ga]["label"]
    risk_flags.append(f"{lbl} ({lvl}) ")
if neutral_count / max(len(_question_ids(db)), 1) >= 0.3:
    risk_flags.append("中立回答が多い (疲弊・回答回避の可能性) ")

type_flags = {
    "a_state": a_state,
    "integration_state": integration_state,
    "replacement_flags": replacement_flags,
    "risk_flags": risk_flags,
    "responsibility_type": resp_type,
    "g5_active": active_g5,
}

# — 9. return —————
return {
    # 本人向けスコア
    "axis_score": axis_score,
    "preference_score": preference_score, # 25点満点に正規化済み
    "observer_score": observer_score,
    "switching_score": switching_score,
    "integration_index": integration_index,
    "internal_a": internal_a,
    "expressive_a": expressive_a,
    # 内部判定
    "low_evidence_index": low_evidence_index,
    "responsibility": responsibility,
    "g5_flags": g5_flags,
    "type_flags": type_flags,
    # 品質指標
    "neutral_frequency": {
        "total": neutral_count,
        "by_axis": dict(neutral_by_axis),
        "rate": round(neutral_count / max(len(_question_ids(db)), 1), 3),
    },
    "diagnostic_null_count": diagnostic_null_count,
}

# =====
# __main__: 動作確認

```

```

# =====
if __name__ == "__main__":
    import os

    db_path = os.path.join(
        os.path.dirname(os.path.abspath(__file__)),
        "quadmind_scoring_db_v3.json"
    )
    db = load_db(db_path)

    sample_responses = {
        "A-1": "a", "A-2": "a", "A-3": "a", "A-4": "a", "A-5": "a", "A-6": "a", "A-7": "a", "A-
        "B-1": "a", "B-2": "a", "B-3": "a", "B-4": "a", "B-5": "a", "B-6": "a", "B-7": "b", "B-
        "C-1": "b", "C-2": "b", "C-3": "b", "C-4": "b", "C-5": "b", "C-6": "b", "C-7": "c", "C-
        "D-1": "b", "D-2": "b", "D-3": "b", "D-4": "b", "D-5": "b", "D-6": "b", "D-7": "a", "D-
        "OB-1": "b", "OB-2": "b", "OB-3": "b", "OB-4": "b", "OB-5": "b",
        "SW-1": "b", "SW-2": "b", "SW-3": "b", "SW-4": "b", "SW-5": "b",
        "iA-1": "a", "iA-2": "a", "iA-3": "a", "iA-4": "a", "iA-5": "a",
        "eA-1": "d", "eA-2": "d", "eA-3": "d", "eA-4": "c", "eA-5": "c",
        "FZ-1": "c", "FZ-2": "c",
        "DR-1": "c", "DR-2": "c", "DR-3": "c",
        "BR-1": "a", "BR-2": "a", "BR-3": "a",
        "AR-1": "b", "AR-2": "b", "AR-3": "b",
        "AG-1": "d", "AG-2": "d", "AG-3": "d",
        "RV-1": "b", "RV-2": "b", "RV-3": "d",
        "IM-1": "c", "IM-2": "d", "IM-3": "c",
    }

    # バリデーションテスト (一問外す)
    print("=== バリデーションテスト ===")
    missing_test = dict(sample_responses)
    del missing_test["A-1"]
    missing = validate_responses(db, missing_test)
    print(f"未回答検出: {missing}")

    # 通常採点
    print("\n=== QuadMind Scoring Engine v3.3 ===\n")
    result = score(db, sample_responses) # strict=True は固定

    print("■ Axis Score (25点満点) ")
    for ax, sc in result["axis_score"].items():
        print(f"    {ax}: {sc}")

    print("\n■ Preference Score (25点満点・正規化済み) ")
    for ax, ps in sorted(result["preference_score"].items(), key=lambda x: -x[1]):
        print(f"    {ax}: {ps}")

```



```

print("\n■ Low Evidence Index (内部判定) ")
for ax, lei in result["low_evidence_index"].items():
    print(f"    {ax}: {lei}")

print(f"\n■ Observer:    {result['observer_score']}")
print(f"■ Switching:    {result['switching_score']}")
print(f"■ Integration: {result['integration_index']}")
print(f"\n■ Internal A:    {result['internal_a']}")
print(f"■ Expressive A: {result['expressive_a']}")

print("\n■ 責任感タイプ")
r = result["responsibility"]
for k, v in r["scores"].items():
    print(f"    {k}: {v}")
print(f"    → {r['type_label']}")

print("\n■ G5 内部フラグ")
for ga, info in result["g5_flags"].items():
    mark = {"high": "★★", "watch": "★ ", "none": "  "}[info["level"]]
    print(f"    {mark} {ga}: {info['score']} / 25  [{info['level']}] {info['lab']}")

print("\n■ type_flags (構造化判定) ")
tf = result["type_flags"]
print(f"    a_state:          {tf['a_state']}")
print(f"    integration_state: {tf['integration_state']}")
print(f"    replacement_flags: {tf['replacement_flags']}")
print(f"    risk_flags:        {tf['risk_flags']}")
print(f"    responsibility:    {tf['responsibility_type']}")
print(f"    g5_active:         {tf['g5_active']}")

print(f"\n■ Neutral Frequency: {result['neutral_frequency']['total']}件")
print(f"■ Diagnostic Null:    {result['diagnostic_null_count']}件")

```